

Myanmar Sign Language Video Recognition for Emergency Domain

A Complete 3-Hour Masterclass on Deep Learning for Low-Resource SLR

Dr. Ye Kyaw Thu

Lab Leader, Language Understanding Lab., Myanmar
Visiting Professor, Core AI Team, NECTEC, Thailand

<https://sites.google.com/site/yekyawthunlp/>

<https://github.com/ye-kyaw-thtu/MSL4Emergency>

June 2026

Course Overview: 3 Hours, Complete Pipeline

Part 1: Foundation (1 hour)

- ① MSL corpus & challenges
- ② Keypoint extraction with MediaPipe
- ③ Data augmentation strategy
- ④ Dataset & data loading

Part 2: Modeling (1 hour)

- ① BiLSTM with attention
- ② Transformer encoder
- ③ ST-GCN architecture
- ④ Training pipeline

Part 3: Evaluation & Deployment (1 hour)

- ① Cross-validation design
- ② Metrics & analysis
- ③ Single/batch inference
- ④ ONNX export for Windows

Learning Outcomes

- Understand 1-sample-per-class problem
- Implement complete SLR pipeline
- Deploy models for real-world use

The Core Challenge: One Sample Per Class

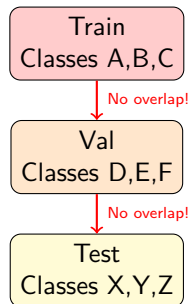
MSL4Emergency Statistics

- **558 videos** total
- **556 unique sign classes**
- **554 classes** have only **1 video** (99.6%)
- Only 2 classes have 2 videos

Why This is Hard

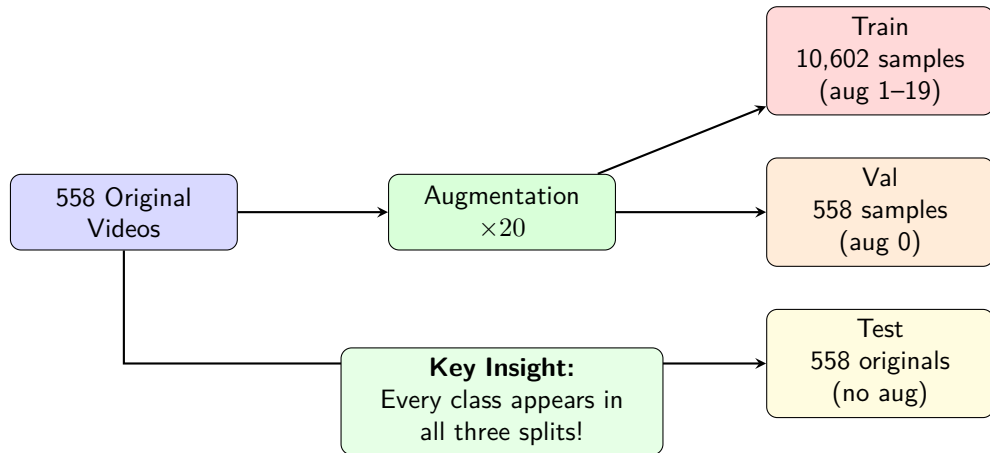
Traditional train/val/test split $\Rightarrow$ **ZERO class overlap**

- Train sees classes A, B, C...
- Test sees classes X, Y, Z...
- Accuracy = 0% (not model failure, **data design error**)



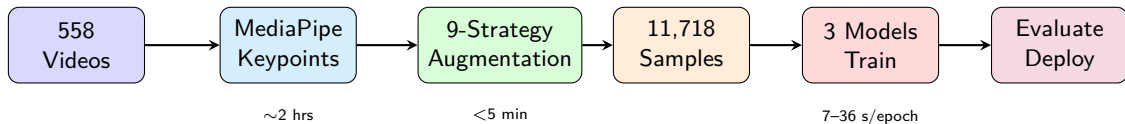
Accuracy = 0%

Our Solution: Augmentation-Indexed Splitting



- **100% class overlap** guaranteed across train/val/test
- Test on **real, un-augmented** keypoints (strictest evaluation)
- Validation sees augmented data (harder, more realistic)

Complete Pipeline Overview



Input:

- .mp4 videos
- annotations.txt
- config.yaml

Intermediate:

- .npy keypoints
- augmented_manifest.json
- checkpoints

Output:

- Trained models
- Evaluation metrics
- ONNX for Windows

Part 1

Data Preparation & Keypoint Extraction

558 Videos → 11,718 Training Samples

Four Emergency Domains

① Fire Safety

မီး (fire), မီးသတ်ဆေးဘူး (extinguisher)

② Flood & Weather

ရေ လွှမ်းမိုးမှု (flooding), ဆူနာမီလှိုင်း (tsunami)

③ Traffic Accidents

ဘေးအန္တရာယ် (danger), ချိုင့် (pothole)

④ Medical Emergencies

အောက်စီဂျင် (oxygen), ချော်လဲ လို့ပါ (I fell)

Property	Value
Total videos	558
Unique classes	556
1 video/class	554 (99.6%)
2 videos/class	2 (0.4%)
Annotation cols	2

Dual Annotation

- **Myanmar text:** Standard written
- **MSL gloss:** Sign order

Annotation Format: Dual Representation

annotations.txt Structure (Tab-Delimited)

Each line has two tab-separated columns:

Col 1 (Normal Myanmar text) `<tab>` **Col 2** (MSL Gloss)

Column 1: Normal Text

Standard Myanmar grammar

မီးသတ်ဆေးဘူး ။

("fire extinguisher")

Column 2: MSL Gloss

Sign language word order

အနီ ဘူးဖြန်း

("red cylinder spray")

Example Pairs

- မီး ။ → မီး
- သဲ အိတ် ။ → အိတ် ထဲ သဲ
- တံခါးပေါက် → တံခါးပေါက်

Annotation Format: Dual Representation

Why Both?

Normal text for end-users; gloss for linguistic analysis & sign-level translation

Repository Structure

```
msl_recognition/  
  data/  
    annotations.txt          # 558 sign annotations  
    videos/                  # 558 .mp4 files  
    keypoints/               # Extracted .npy files  
      idx20-1.npy  
      idx20-1_vis.npy  
      extraction_stats.json  
    augmented/               # Augmented data  
      train/                 # 10,602 files  
      val/                   # 558 files  
      augmented_manifest.json  
    label_map.json           # Class vocabulary  
    splits.json              # Train/val/test indices  
src/                         # Python source code  
  extract_keypoints.py  
  augment.py  
  dataset.py  
  models/  
scripts/                     # Shell scripts (00-09)  
config/config.yaml  
results/                     # Experiments & checkpoints
```

Environment Setup

Python Environment

```
1 # Create virtual environment
2 python -m venv venv
3 source venv/bin/activate
4
5 # Install dependencies
6 pip install -r requirements.txt
```

Key Libraries

- torch>=2.1.0 (CUDA 11.8+)
- mediapipe>=0.10.7
- opencv-python>=4.8.0
- numpy, scipy, scikit-learn
- transformers (for comparison)

Hardware Used

- **GPU:** NVIDIA RTX 3090 Ti (24GB)
- **CPU:** Intel Core i9-12900
- **RAM:** 64GB recommended

MediaPipe Note

Holistic removed in v0.10.13+

We use **separate Pose + Hands**

```
from mediapipe.solutions import pose
from mediapipe.solutions import hands
```

Script 01: Prepare Data

```
1 #!/usr/bin/env bash
2 # 01_prepare_data.sh - Parse annotations, build vocab, create splits
3 DATA_DIR="data"
4 VIDEO_DIR="${DATA_DIR}/videos"
5 ANNOTATION_FILE="${DATA_DIR}/annotations.txt"
6 CONFIG="config/config.yaml"
7
8 python src/prepare_data.py \
9     --config "${CONFIG}" \
10    --verify_keypoints
```

Outputs:

- data/label_map.json – Class vocabulary (556 classes)
- data/splits.json – Train/val/test indices
- data/kfold_splits.json – 5-fold CV splits

Run Once

This script parses annotations and creates splits. Run **before** any training.

Keypoint Extraction: Overview

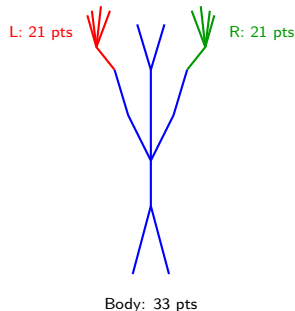
What We Extract Per Frame

$$\mathbf{f}_t = \left[\underbrace{\mathbf{p}_t}_{33 \times 3}, \underbrace{\mathbf{l}_t}_{21 \times 3}, \underbrace{\mathbf{r}_t}_{21 \times 3} \right] \in \mathbb{R}^{225}$$

- $\mathbf{p}_t$: 33 body pose landmarks
- $\mathbf{l}_t$: 21 left hand landmarks
- $\mathbf{r}_t$: 21 right hand landmarks
- Total: 75 nodes $\times$ 3 coords = 225 features

Output Files

- `idx20-X.npy`: Coordinates ($T, 75, 3$)
- `idx20-X_vis.npy`: Visibility ($T, 33$)



MediaPipe Import: Compatibility Fix

Problem

MediaPipe removed `solutions.holistic` in v0.10.13+ (Python 3.12)

```
1 def _import_mp_modules():
2     """Try all known import paths; return (pose, hands, drawing)."""
3     import importlib
4     prefixes = ["mediapipe.solutions", "mediapi.python.solutions"]
5     for prefix in prefixes:
6         try:
7             pose = importlib.import_module(f"{prefix}.pose")
8             hands = importlib.import_module(f"{prefix}.hands")
9             draw = importlib.import_module(f"{prefix}.drawing_utils")
10            logging.info(f"[mediapipe] loaded via '{prefix}'")
11            return pose, hands, draw
12        except (ImportError, ModuleNotFoundError):
13            continue
14    # Legacy fallback for mediapipe <= 0.9
15    import mediapipe as mp
16    return mp.solutions.pose, mp.solutions.hands, mp.solutions.drawing_utils
```

extract_keypoints.py - Import Helper

Best Practice

Use `pip install mediapipe==0.10.18` for stability

Important: Hand Label Convention

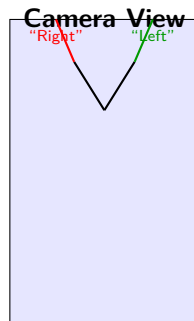
Camera vs. Signer Perspective

MediaPipe reports handedness from **camera** view (mirrored):

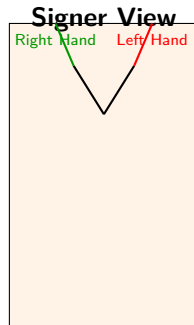
- Camera “Right” = Signer’s **LEFT** hand
- Camera “Left” = Signer’s **RIGHT** hand

```
1 def extract_frame_keypoints(pose_res, hands_res):
2     for lm_list, handedness in zip(
3         hands_res.multi_hand_landmarks,
4         hands_res.multi_handedness,
5     ):
6         label = handedness.classification[0].label
7         arr = _lm_to_array(lm_list, HAND_N)
8         if label == 'Right':
9             lh = arr # camera-right = signer's LEFT
10        else:
11            rh = arr # camera-left = signer's RIGHT
12    return np.concatenate([pose, lh, rh], axis=0)
```

`extract_frame_keypoints()`



Swap
→



Normalization: Shoulder-Based

Why Normalize?

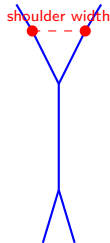
Raw coordinates depend on:

- Camera distance
- Body position in frame
- Signer height/size

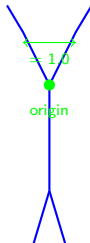
```
1 def normalize_sequence(seq: np.ndarray) -> np.ndarray:  
2     l_sh = seq[:, POSE_LEFT_SHOULDER, :]  
3     r_sh = seq[:, POSE_RIGHT_SHOULDER, :]  
4     mid = (l_sh + r_sh) / 2.0  
5     dist = np.linalg.norm(r_sh - l_sh,  
6                           axis=1, keepdims=True)  
7     dist = np.where(dist < 1e-6, 1.0, dist)  
8     return ((seq - mid[:, None, :]) /  
9            dist[:, None, :]).astype(np.float32)
```

normalize_sequence()

Before



After



Script 02: Extract Keypoints

```
1 #!/usr/bin/env bash
2 # 02_extract_keypoints.sh
3 VIDEO_DIR="data/videos"
4 KEYPOINT_DIR="data/keypoints"
5 COMPLEXITY=2 # 0=fastest, 2=most accurate
6
7 python src/extract_keypoints.py \
8     --video_dir "${VIDEO_DIR}" \
9     --output_dir "${KEYPOINT_DIR}" \
10    --complexity "${COMPLEXITY}"
```

Output: extraction_stats.json

```
{
  "total": 558, "processed": 557, "failed": 0,
  "seq_len_min": 50, "seq_len_max": 387,
  "seq_len_mean": 130.57,
  "lh_mean_presence": 0.618, "rh_mean_presence": 0.713
}
```

Timing

558 videos on RTX 3090 Ti: ~1.5–2 hours

Understanding .npy Files

Coordinate File: idx20-X.npy

```
import numpy as np
coords = np.load('data/keypoints/idx20-401.npy')
print(coords.shape) # (88, 75, 3)
print(coords[0, :3, :])
# [[-0.012 -0.990 -1.182]
#  [ 0.045 -1.169 -1.063]
#  [ 0.084 -1.167 -1.066]]
```

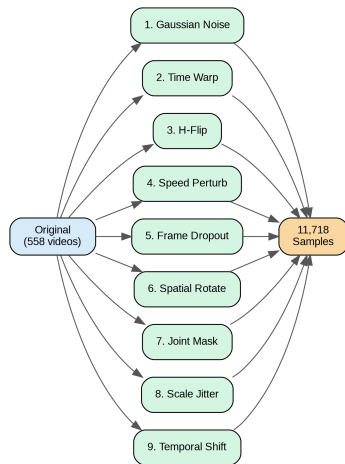
Visibility File: idx20-X_vis.npy

```
vis = np.load('data/keypoints/idx20-401_vis.npy')
print(vis.shape) # (88, 33)
print(vis[0, :3])
# [0.995 0.983 0.984]
```

Why Separate?

- **Coordinates:** Primary feature vector for model input
- **Visibility:** Can filter/mask unreliable landmarks

Data Augmentation: The Key Innovation



9 Augmentation Strategies

Strategy	Prob.	Parameter	Simulates
Gaussian Noise	0.70	$\sigma = 0.008$	Detector jitter
Time Warp	0.60	$\sigma = 0.15$, 4 knots	Rhythm variation
Horizontal Flip	0.50	—	Handedness invariance
Speed Perturbation	0.60	factor $\in [0.75, 1.35]$	Signing speed
Frame Dropout	0.40	$\leq 10\%$ frames	Detector failures
Spatial Rotation	0.50	$\pm 12^\circ$	Off-axis camera
Joint Masking	0.30	$\leq 10\%$ joints	Occlusion
Scale Jitter	0.40	factor $\in [0.85, 1.15]$	Size variation
Temporal Shift	0.30	≤ 5 frames	Boundary variation

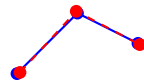
Each strategy applied independently

Combinations arise naturally $\Rightarrow$ highly varied synthetic samples

Augmentation 1: Gaussian Noise

```
1 def augment_gaussian_noise(  
2     seq: np.ndarray,  
3     std: float = 0.008  
4 ) -> np.ndarray:  
5     noise = np.random.normal(0, std, seq.shape)  
6     return (seq + noise).astype(np.float32)
```

augment_gaussian_noise()



Original

Noisy ($\sigma = 0.008$)

Effect

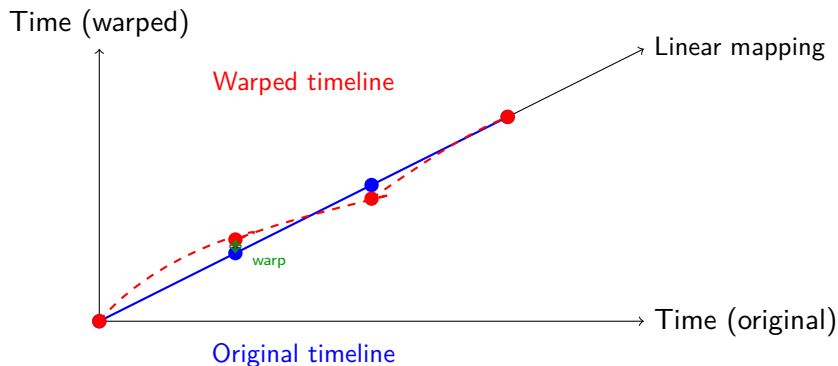
- Adds small random offsets to (x, y, z)
- Simulates MediaPipe detection uncertainty
- $\sigma = 0.008$ is subtle but effective

Augmentation 2: Time Warp

```
1 def augment_time_warp(seq: np.ndarray, sigma: float = 0.15,  
2                       knot: int = 4) -> np.ndarray:  
3     T = seq.shape[0]  
4     if T < 4:  
5         return seq  
6     orig_steps = np.linspace(0, T - 1, num=knot + 2)  
7     warp_steps = orig_steps + np.random.normal(  
8         0, sigma * T, orig_steps.shape)  
9     warp_steps[0], warp_steps[-1] = 0, T - 1  
10    warp_steps = np.clip(np.sort(warp_steps), 0, T - 1)  
11    cs = CubicSpline(warp_steps, orig_steps)  
12    new_steps = np.linspace(0, T - 1, T)  
13    orig_times = np.clip(cs(new_steps), 0, T - 1)  
14    warped = np.zeros_like(seq)  
15    for j in range(seq.shape[1]):  
16        for c in range(seq.shape[2]):  
17            warped[:, j, c] = np.interp(  
18                orig_times, np.arange(T), seq[:, j, c])  
19    return warped.astype(np.float32)
```

augment_time_warp()

Time Warp: Visualization



- Simulates **rhythm variation** in signing
- Same sign, different timing patterns
- 4 knots provide smooth, realistic warps

Augmentation 3: Horizontal Flip

```
1 def augment_horizontal_flip(seq: np.ndarray) -> np.ndarray:
2     flipped = seq.copy()
3     flipped[:, :, 0] = -flipped[:, :, 0]
4     for i, j in POSE_MIRROR_PAIRS:
5         flipped[:, [i, j], :] = flipped[:, [j, i], :]
6     lh = flipped[:, LEFT_HAND_START:LEFT_HAND_START+HAND_N, :].copy()
7     rh = flipped[:, RIGHT_HAND_START:RIGHT_HAND_START+HAND_N, :].copy()
8     flipped[:, LEFT_HAND_START:LEFT_HAND_START+HAND_N, :] = rh
9     flipped[:, RIGHT_HAND_START:RIGHT_HAND_START+HAND_N, :] = lh
10    return flipped.astype(np.float32)
```

augment_horizontal_flip()

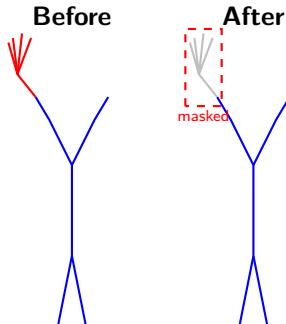
Purpose

Promotes **handedness invariance** – model learns sign from either perspective

Augmentation 7: Joint Masking (Occlusion)

```
1 def augment_joint_mask(  
2     seq: np.ndarray,  
3     max_mask_ratio: float = 0.10  
4 ) -> np.ndarray:  
5     N = seq.shape[1]  
6     n_mask = max(1, int(N * np.random.uniform(  
7         0, max_mask_ratio)))  
8     indices = random.sample(range(N), n_mask)  
9     out = seq.copy()  
10    out[:, indices, :] = 0.0  
11    return out.astype(np.float32)
```

augment_joint_mask()

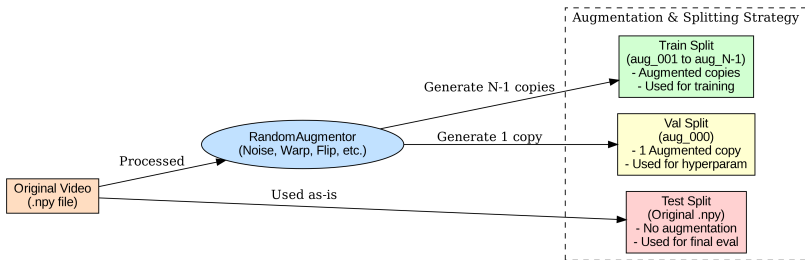


Combined Random Augmentor

```
1 class RandomAugmentor:
2     """Apply a random subset with individual probs."""
3     def __init__(self, cfg: dict):
4         self.cfg = cfg
5     def _p(self, key: str) -> float:
6         return self.cfg.get(key, {}).get('prob', 0.0)
7     def __call__(self, seq: np.ndarray) -> np.ndarray:
8         aug = seq.copy()
9         if random.random() < self._p('gaussian_noise'):
10             aug = augment_gaussian_noise(aug)
11         if random.random() < self._p('time_warp'):
12             aug = augment_time_warp(aug)
13         if random.random() < self._p('horizontal_flip'):
14             aug = augment_horizontal_flip(aug)
15         if random.random() < self._p('speed_perturbation'):
16             aug = augment_speed_perturbation(aug)
17         if random.random() < self._p('frame_dropout'):
18             aug = augment_frame_dropout(aug)
19         if random.random() < self._p('spatial_rotate'):
20             aug = augment_spatial_rotate(aug)
21         if random.random() < self._p('joint_mask'):
22             aug = augment_joint_mask(aug)
23         if random.random() < self._p('scale_jitter'):
24             aug = augment_scale_jitter(aug)
25         if random.random() < self._p('temporal_shift'):
26             aug = augment_temporal_shift(aug)
27         return aug
```

RandomAugmentor class

Augmentation-Indexed Split Design



Train

19 copies $\times$ 558
= 10,602 samples

Validation

1 copy $\times$ 558
= 558 samples

Test

Originals only
= 558 samples

100% Class Overlap Guaranteed

Every one of 556 classes appears in train, val, AND test

Augmented Manifest Structure

```
1 {
2   "train": [
3     {
4       "idx": 0,
5       "label": "<Myanmar text label>",
6       "video_path": "data/videos/idx20-1.mp4",
7       "keypoint_path": "data/augmented/train/0000_aug001.npy",
8       "is_augmented": true,
9       "aug_id": 1,
10      "split": "train"
11    }
12  ],
13  "val": [
14    {
15      "idx": 0,
16      "label": "<Myanmar text label>",
17      "keypoint_path": "data/augmented/val/0000_aug000.npy",
18      "is_augmented": true,
19      "aug_id": 0,
20      "split": "val"
21    }
22  ],
23  "test": [
24    {
25      "idx": 0,
26      "label": "<Myanmar text label>",
27      "keypoint_path": "data/keypoints/idx20-1.npy",
28      "is_augmented": false,
29      "split": "test"
30    }
31  ]
32 }
```

Note: The `label` field contains the actual Myanmar text (e.g., မိုး ။) from `annotations.txt`.

Script 03: Augment Data

```
1 #!/usr/bin/env bash
2 # 03_augment_data.sh - All-class augmentation
3 CONFIG="config/config.yaml"
4 AUG_FACTOR=20
5 python src/augment.py \
6     --config "${CONFIG}" \
7     --aug_factor "${AUG_FACTOR}"
```

Output

```
=====
Augmentation Complete (all-class design)
=====
Original videos : 558
Aug factor      : 20 (train uses 19, val uses 1)
Train samples   : 10602 (augmented, all classes)
Val samples     : 558 (augmented, all classes)
Test samples    : 558 (original, all classes)
Class overlap   : 100% across all splits
=====
```

Timing

558 videos $\times$ 20 copies: <5 minutes

MSL Dataset Class

```
1 class MSLDataset(Dataset):
2     """Returns dict with keypoints, label, length, mask."""
3     def __init__(self, records, label2idx, max_seq_len=200,
4                 flatten=True, augmentor=None):
5         self.records = records
6         self.label2idx = label2idx
7         self.max_seq_len = max_seq_len
8         self.flatten = flatten # (T,75,3) -> (T,225)
9         self.augmentor = augmentor
10
11     def __getitem__(self, idx):
12         rec = self.records[idx]
13         seq = np.load(rec['keypoint_path']).astype(np.float32)
14         if self.augmentor:
15             seq = self.augmentor(seq)
16         T = seq.shape[0]
17         if T > self.max_seq_len:
18             start = (T - self.max_seq_len) // 2
19             seq = seq[start:start + self.max_seq_len]
20             T = self.max_seq_len
21         # Zero-pad if too short
22         if T < self.max_seq_len:
23             pad = np.zeros((self.max_seq_len - T, *seq.shape[1:]),
24                           dtype=np.float32)
25             seq = np.concatenate([seq, pad], axis=0)
26         mask = np.zeros(self.max_seq_len, dtype=bool)
27         mask[T:] = True # True = padded position
28         if self.flatten:
29             seq = seq.reshape(self.max_seq_len, -1) # (T, 225)
30         return {
31             'keypoints': torch.from_numpy(seq),
```

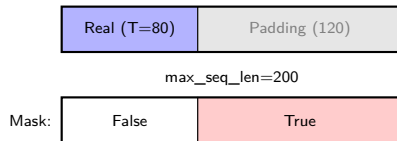
Padding and Masking

Why Pad to Fixed Length?

- Batch processing requires uniform tensor shapes
- Videos have different lengths (50–387 frames)
- We pad/crop to `max_seq_len=200`

Padding Strategy

- **Too long:** Centre-crop
- **Too short:** Zero-pad at end
- **Mask:** Boolean tensor, True where padded



Mask Usage

- **BiLSTM:** Pack padded sequence
- **Transformer:** `key_padding_mask` in attention

Part 2

Model Architectures & Training

BiLSTM · Transformer · ST-GCN

Three Model Architectures

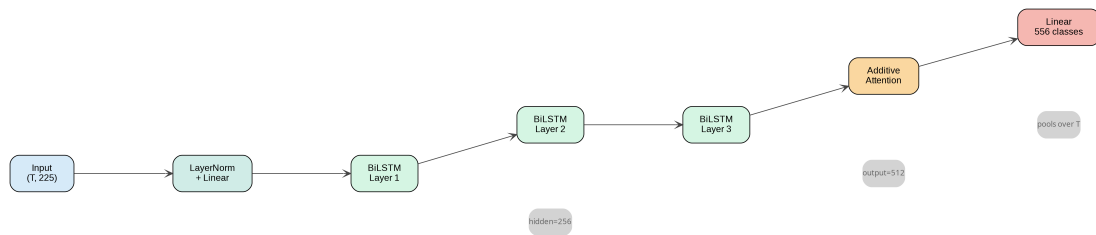
Property	BiLSTM	Transformer	ST-GCN
Input shape	$(T, 225)$	$(T, 225)$	$(3, T, 75)$
Hidden dim	256	256	64→256
Layers	3 BiLSTM	4 Enc. blocks	4 GCN blocks
Attention	Additive	8-head MH	—
Parameters	4.8M	3.4M	2.8M
Epoch time	7 s	4 s	36 s
Val Top-1	95.31%	96.88%	94.44%
Test Top-1	100%	100%	100%
CV Mean±Std	88.69±0.88	95.93±0.27	93.34±0.27

Key Finding

Transformer: Best accuracy, most efficient

ST-GCN: Competitive, most compact, graph structure helps

BiLSTM Architecture



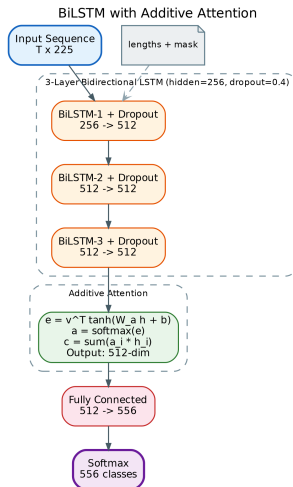
Key Features:

- 3-layer bidirectional LSTM
- Hidden size 256 $\rightarrow$ output 512
- Bahdanau additive attention
- Forget gate bias = 1.0

Parameters: 4.8M

- Embedding: $225 \times 256 = 57K$
- $3 \times$ BiLSTM: $\sim 4M$
- Attention + Classifier: $\sim 0.7M$

BiLSTM Architecture



BiLSTM: Code Structure

```
1 class BiLSTMClassifier(nn.Module):
2     def __init__(self, input_dim=225, hidden_dim=256, num_classes=556,
3                 num_layers=3, dropout=0.4):
4         super().__init__()
5         self.input_proj = nn.Linear(input_dim, hidden_dim)
6         self.ln = nn.LayerNorm(hidden_dim)
7         self.lstm = nn.LSTM(
8             hidden_dim, hidden_dim, num_layers=num_layers,
9             batch_first=True, bidirectional=True, dropout=dropout)
10        for name, param in self.lstm.named_parameters():
11            if 'bias_ih' in name:
12                nn.init.constant_(
13                    param[hidden_dim:2*hidden_dim], 1.0)
14        self.attention = AdditiveAttention(hidden_dim * 2)
15        self.dropout = nn.Dropout(dropout)
16        self.fc = nn.Linear(hidden_dim * 2, num_classes)
17
18    def forward(self, x, lengths=None, mask=None):
19        x = self.ln(self.input_proj(x))
20        x = self.dropout(x)
21        if lengths is not None:
22            x = pack_padded_sequence(x, lengths,
23                                    batch_first=True, enforce_sorted=False)
24        out, _ = self.lstm(x)
25        if lengths is not None:
26            out, _ = pad_packed_sequence(
27                out, batch_first=True)
28        context, _ = self.attention(out, mask=mask)
29        logits = self.fc(self.dropout(context))
30        return logits
```

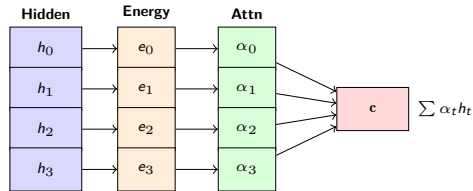
Additive (Bahdanau) Attention

Formulation

$$\mathbf{e}_t = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{h}_t) \quad (1)$$

$$\alpha = \text{softmax}(\mathbf{e}) \quad (2)$$

$$\mathbf{c} = \sum_t \alpha_t \mathbf{h}_t \quad (3)$$



```
1 class AdditiveAttention(nn.Module):
2     def __init__(self, hidden_dim):
3         super().__init__()
4         self.W = nn.Linear(hidden_dim, hidden_dim)
5         self.v = nn.Linear(hidden_dim, 1)
6     def forward(self, hidden, mask=None):
7         energy = torch.tanh(self.W(hidden))
8         energy = self.v(energy).squeeze(-1)
9         if mask is not None:
10             energy = energy.masked_fill(
11                 mask, float('-inf'))
12         alpha = F.softmax(energy, dim=-1)
13         context = (alpha.unsqueeze(-1) * hidden).sum(1)
14         return context, alpha
```

Transformer Encoder Architecture

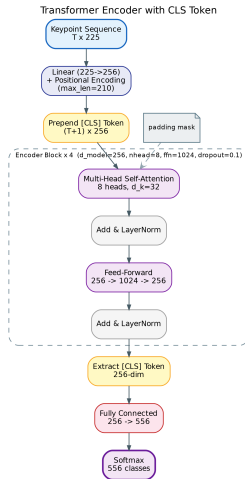
Key Features:

- Learnable [CLS] token prepended
- Sinusoidal positional encoding
- 4 Pre-LayerNorm encoder blocks
- 8-head self-attention ($d_k=32$)

Parameters: 3.4M

- Embedding: $225 \times 256 = 57K$
 - 4× Encoder: $\sim 2.8M$
- CLS + Classifier: $\sim 0.5M$

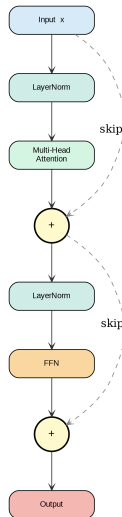
Transformer Encoder Architecture



Transformer Encoder Block

```
1 class EncoderBlock(nn.Module):
2     def __init__(self, d_model=256, nhead=8,
3                   d_ff=1024, dropout=0.1):
4         super().__init__()
5         self.norm1 = nn.LayerNorm(d_model)
6         self.attn = nn.MultiheadAttention(
7             d_model, nhead, dropout=dropout,
8             batch_first=True)
9         self.norm2 = nn.LayerNorm(d_model)
10        self.ffn = nn.Sequential(
11            nn.Linear(d_model, d_ff),
12            nn.GELU(),
13            nn.Dropout(dropout),
14            nn.Linear(d_ff, d_model),
15            nn.Dropout(dropout),
16        )
17    def forward(self, x, mask=None):
18        x_norm = self.norm1(x)
19        attn_out, _ = self.attn(
20            x_norm, x_norm, x_norm,
21            key_padding_mask=mask)
22        x = x + attn_out
23        x = x + self.ffn(self.norm2(x))
24        return x
```

EncoderBlock (Pre-LayerNorm)



ST-GCN: Skeleton Graph Structure

75 Nodes Total

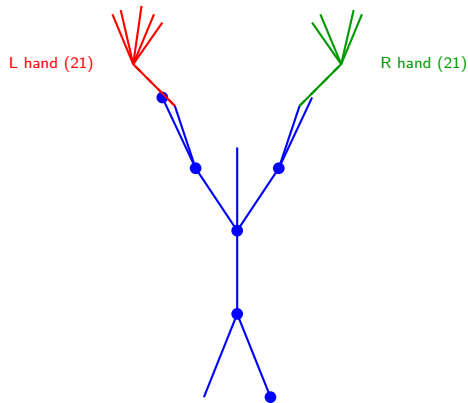
- 33 body pose (MediaPipe)
- 21 left hand
- 21 right hand

Edge Types

- 1 **Spatial**: Body joints (MediaPipe topology)
- 2 **Spatial**: Hand joints (finger bones)
- 3 **Cross-body**: Wrist $\rightarrow$ hand root

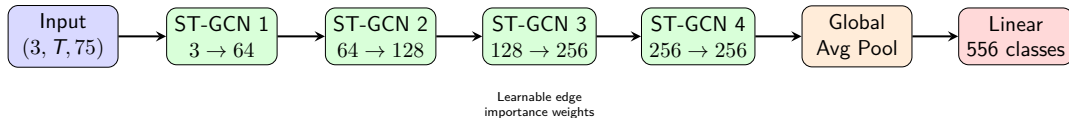
Input Shape

$(3, T, 75)$: 3 channels, T frames, 75 nodes



75 Nodes: 33 body + 21×2 hands

ST-GCN Architecture



Key Features:

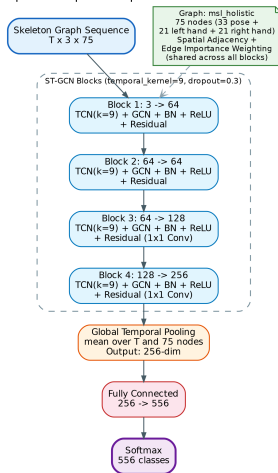
- 4 ST-GCN blocks
- Channel progression: $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$
- Learnable edge importance
- Normalized adjacency: $\tilde{A} = D^{-1/2}(A + I)D^{-1/2}$

Parameters: 2.8M

- $4 \times$ ST-GCN: $\sim 2.5M$
- Classifier: $\sim 0.1M$
- Most compact model

ST-GCN Architecture

Spatial-Temporal Graph Convolutional Network

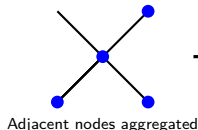


ST-GCN Block: Spatial-Temporal Convolution

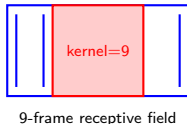
```
1 class STGCNBlock(nn.Module):
2     def __init__(self, in_ch, out_ch, A, stride=1,
3                   dropout=0.3):
4         super().__init__()
5         self.A = A
6         self.edge_w = nn.Parameter(
7             torch.ones(A.shape[1]))
8         self.gcn = nn.Conv2d(
9             in_ch, out_ch, kernel_size=1)
10        self.tcn = nn.Sequential(
11            nn.Conv2d(out_ch, out_ch,
12                      kernel_size=(9, 1),
13                      padding=(4, 0)),
14            nn.BatchNorm2d(out_ch),
15            nn.Dropout(dropout),
16        )
17        self.relu = nn.ReLU()
18    def forward(self, x):
19        A_w = torch.einsum(
20            'nv,cn->cvn', self.A, self.edge_w)
21        x = torch.einsum('nvc,bctv->bntv', A_w, x)
22        x = self.gcn(x)
23        x = self.tcn(x)
24        return self.relu(x)
```

STGCNBlock

Spatial Graph Conv



Temporal Conv (TCN)



Training Configuration

Hyperparameter	Value
Optimizer	AdamW
Learning rate	5×10^{-4}
Weight decay	10^{-4}
LR scheduler	Cosine annealing
Warmup epochs	10
Batch size	32
Max epochs	150
Early stopping patience	25
Label smoothing	0.1
Gradient clip norm	1.0
Mixed precision (AMP)	Enabled
Max sequence length	200 frames
Hardware	
GPU	NVIDIA RTX 3090 Ti (24 GB)

Loss Function: Cross-Entropy with Class Weights

```
1 def compute_class_weights(records, label2idx, device):
2     counts = np.zeros(len(label2idx))
3     for rec in records:
4         counts[label2idx[rec['label']]] += 1
5     weights = 1.0 / (counts + 1e-6)
6     weights = weights / weights.sum() * len(label2idx)
7     return torch.tensor(weights, dtype=torch.float32,
8                           device=device)
```

Class weight computation

```
1 class_weights = compute_class_weights(
2     train_records, label2idx, device
3 )
4 criterion = nn.CrossEntropyLoss(
5     weight=class_weights,
6     label_smoothing=0.1
7 )
```

Loss initialization

Why Class Weights?

- Minor class imbalance (2 classes have 2 samples)
- Inverse frequency gives more weight to rare classes
- Label smoothing (0.1) prevents overconfidence

Alternative: Focal Loss

For severe imbalance:

```
criterion = FocalLoss(
    gamma=2.0, weight=class_weights,
    label_smoothing=0.1
)
```

Training Loop Structure

```
1 def train_one_epoch(model, loader, optimizer, criterion,  
2                     scaler, device, cfg):  
3     model.train()  
4     total_loss, correct, total = 0, 0, 0  
5     for batch in tqdm(loader, desc="Training"):  
6         kp = batch['keypoints'].to(device, non_blocking=True)  
7         labels = batch['label'].to(device, non_blocking=True)  
8         mask = batch['mask'].to(device, non_blocking=True)  
9         lengths = batch['length'].to(device, non_blocking=True)  
10        optimizer.zero_grad()  
11        with torch.amp.autocast('cuda',  
12                                enabled=cfg['training']['use_amp']):  
13            if model_type == 'bilstm':  
14                logits = model(kp, lengths=lengths, mask=mask)  
15            elif model_type == 'transformer':  
16                logits = model(kp, mask=mask)  
17            else:  
18                logits = model(kp)  
19            loss = criterion(logits, labels)  
20            scaler.scale(loss).backward()  
21            scaler.unscale_(optimizer)  
22            torch.nn.utils.clip_grad_norm_  
23                (model.parameters(), max_norm=1.0)  
24            scaler.step(optimizer)  
25            scaler.update()  
26            total_loss += loss.item() * labels.size(0)  
27            correct += (logits.argmax(1) == labels).sum().item()  
28            total += labels.size(0)  
29    return {'loss': total_loss / total,  
30            'top1': correct / total}
```

train_one_epoch()

Script 04: Train Model

```
1 #!/usr/bin/env bash
2 # 04_train.sh
3 CONFIG="config/config.yaml"
4 MODEL="bilstm"
5 EXP="exp_bilstm"
6 python src/train.py \
7     --config "${CONFIG}" \
8     --model "${MODEL}" \
9     --exp "${EXP}"
```

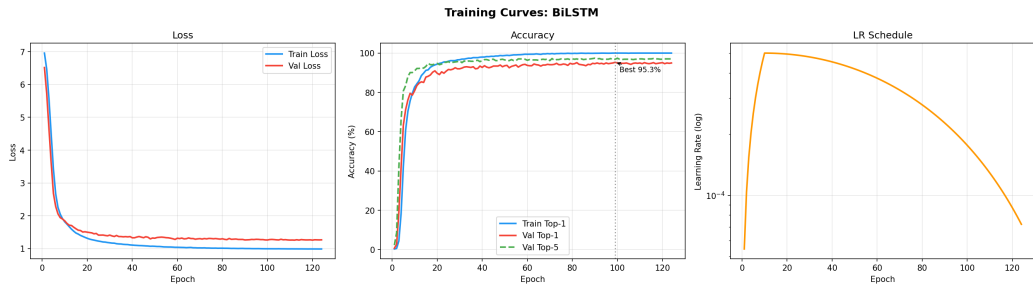
Output Structure

```
results/exp_bilstm/
  checkpoints/
    best.pth           # Best model checkpoint
  history.json         # Training curves
  config.yaml          # Saved config
  train.log            # Training log
```

Checkpoint Contents

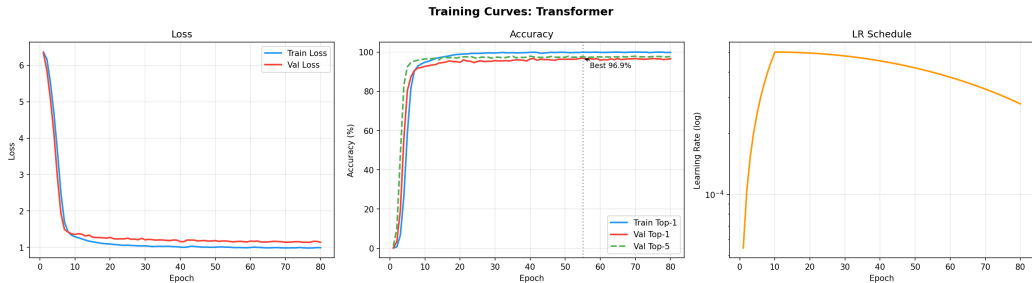
```
epoch, model_type, state_dict, val_top1, num_classes, config
```


Training Curves: BiLSTM



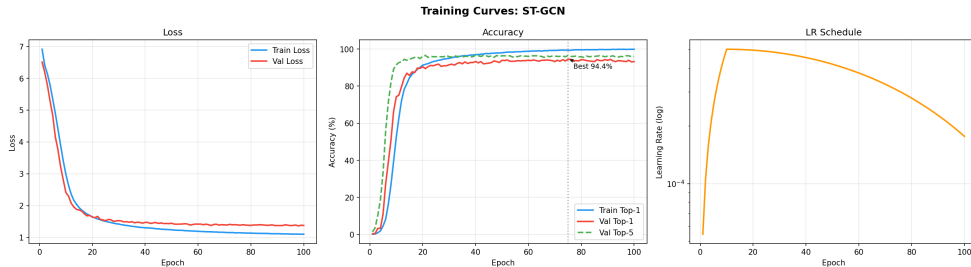
128 epochs, 7 s/epoch | Validation: 95.31%

Training Curves: Transformer



82 epochs, 4 s/epoch | Validation: 96.88%

Training Curves: ST-GCN



101 epochs, 36 s/epoch | Validation: 94.44%

Observations (All Models)

- All models converge smoothly without instability
- Transformer: Fastest convergence, best accuracy
- ST-GCN: Slowest per epoch but highly stable

Part 3

Evaluation, Cross-Validation & Deployment

Metrics · K-Fold CV · Inference · ONNX

Why K-Fold Cross-Validation?

Single Split Limitations

- Small dataset $\Rightarrow$ high variance
- One “lucky” or “unlucky” split
- Cannot assess stability

K-Fold Solution

- Train/val on K different partitions
- Report mean $\pm$ std
- Test set fixed (558 originals)



Folding over 10,602 augmented training samples

Cross-Validation: Correct Design

Wrong: Fold Over 558 Originals

- 5-fold $\Rightarrow$ ~ 446 train, ~ 112 val **ZERO class overlap** $\Rightarrow$ 0% accuracy every fold

Correct: Fold Over 10,602 Augmented Samples

- Each class has 19 augmented copies in train pool
- Every fold: train $\cap$ val contains **all 556 classes**
- Test set: 558 originals (fixed across all folds)

Fold Structure (K=5):

Train: $4/5 \times 10,602 \approx 8,481$ samples (all 558 classes)

Val: $1/5 \times 10,602 \approx 2,121$ samples (all 558 classes)

Test: 558 originals (same across all folds)

5-Fold Cross-Validation Results

Model	F0	F1	F2	F3	F4	Val	Test
	Validation Accuracy (%)					Mean±Std	Mean±Std
BiLSTM	88.4	90.3	88.1	88.9	87.8	88.69±0.88	99.65±0.11
Transformer	96.3	95.9	95.9	95.5	96.0	95.93±0.27	99.97±0.07
ST-GCN	93.1	93.9	93.3	93.3	93.1	93.34±0.27	99.97±0.07

Key Observations:

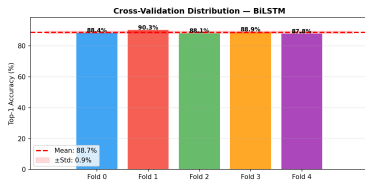
- Transformer: Highest mean, low variance
- ST-GCN: Same variance as Transformer
- BiLSTM: Higher variance (0.88%)

Test vs. Val Gap:

- Test > Val (expected by design)
- Test uses **real** keypoints
- Val uses **augmented** (harder)

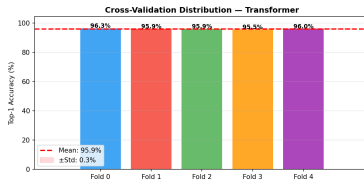
Cross-Validation: Accuracy Distributions

BiLSTM



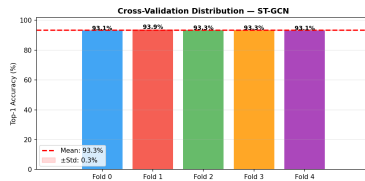
88.7% $\pm$ 0.9%

Transformer



95.9% $\pm$ 0.3%

ST-GCN



93.3% $\pm$ 0.3%

Stability Analysis

- Transformer & ST-GCN: Very low variance (0.27%)
- BiLSTM: Higher variance – more sensitive to partition

Comprehensive Evaluation Metrics

Model	Params (M)	Val Top-1	Test Top-1	Test Top-5	Test F1
BiLSTM	4.8	95.31%	100.00%	100.00%	100.00%
Transformer	3.4	96.88%	100.00%	100.00%	100.00%
ST-GCN	2.8	94.44%	100.00%	100.00%	100.00%

Metrics Explained

- **Top-1:** Best prediction correct?
- **Top-5:** Correct in top 5?
- **F1:** Harmonic mean of P & R
- **Macro:** Average over classes

Why 100% on Test?

- Test = original keypoints (no augmentation)
- Model trained on augmented variants
- Real data is “easier” than augmented
- This is **by design**, not overfitting

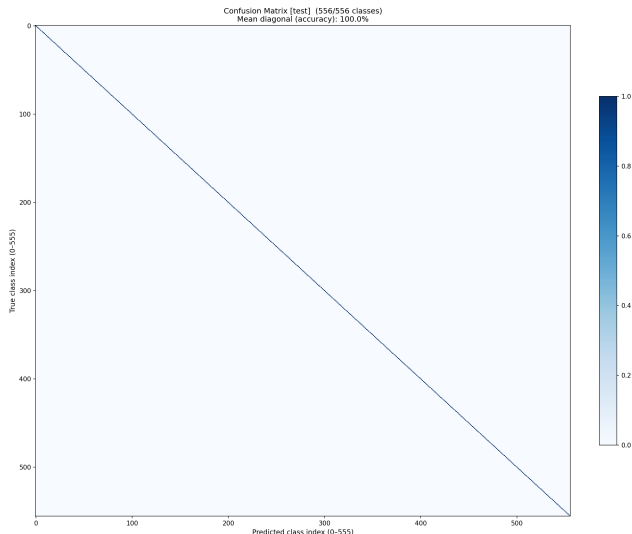
Script 05: Evaluate Model

```
1 #!/usr/bin/env bash
2 # 05_evaluate.sh
3 CHECKPOINT="results/exp_bilstm/checkpoints/best.pth"
4 CONFIG="config/config.yaml"
5 SPLIT="test"
6 python src/evaluate.py \
7     --checkpoint "${CHECKPOINT}" \
8     --config "${CONFIG}" \
9     --split "${SPLIT}"
```

Outputs

```
results/exp_bilstm/evaluation/
metrics_test.json           # Summary metrics
per_class_test.csv          # Per-class P/R/F1
confusion_matrix_test.npy   # Raw CM
confusion_matrix_test.png   # Visualized CM
predictions_test.csv        # All predictions
```

Confusion Matrix Analysis



Interpretation

- **Diagonal:** Correct predictions
- **Off-diagonal:** Confusions
- Row-normalized: Per-class recall

Common Confusion Patterns

- 1 Similar hand shape, different movement
- 2 Medical signs with fingerspelling
- 3 Water-related signs

Top Classes by F1 (Test)

- မီးချိတ် – $F1=1.000$, $P=1.000$, $R=1.000$
- သဲ အိတ် ။ – $F1=1.000$, $P=1.000$, $R=1.000$
- မီးသတ်ဆေးဘူး – $F1=1.000$, $P=1.000$, $R=1.000$
- အောက်စီဂျင် – $F1=1.000$, $P=1.000$, $R=1.000$

Bottom Classes by F1 (Validation)

- ပေါက်တိုး လွှမ်း – $F1=0.833$
- အောက်စီဂျင် ။ – $F1=0.857$

Observations

- Test set: All classes $F1=1.0$ (100%)
- Validation set: Some classes confused due to augmentation
- Bottom classes often involve subtle hand movements

Inference: Three Modes

Single Video



- Input: .mp4 file
- Output: Top-K predictions
- Use: Testing, demo

Batch Processing



- Input: Directory
- Output: CSV file
- Use: Bulk evaluation

Webcam Live



- Input: Camera stream
- Output: Overlay text
- Use: Real-time demo

Single Video Inference: Code

```
1 def infer_video(video_path, model, model_type, cfg, idx2label,  
2                 device, top_k=5):  
3     # 1. Extract keypoints  
4     pose_det, hands_det = _open_detectors(model_complexity=2)  
5     result = process_video(video_path,  
6                             detectors={'pose': pose_det, 'hands': hands_det},  
7                             normalize=True)  
8     seq = result['keypoints']  
9     # 2. Preprocess for model  
10    tensor = preprocess_sequence(  
11        seq, cfg['data']['max_seq_len'], model_type)  
12    real_len = min(seq.shape[0], cfg['data']['max_seq_len'])  
13    # 3. Predict  
14    preds = predict_tensor(model, tensor, model_type, real_len,  
15                           cfg['data']['max_seq_len'], device, top_k)  
16    # 4. Display results  
17    for rank, (idx, prob) in enumerate(preds, start=1):  
18        label = idx2label.get(idx, str(idx))  
19        print(f"[rank]. [{prob*100:5.1f}%] {label}")  
20    return preds
```

infer.py - Key Functions

Inference Demo: Sample Results

Video : idx20-30.mp4

Frames: 127 FPS: 30.0

Hands : L=69% R=65%

Top-5 predictions:

1. [91.0%] 
2. [0.1%]
3. [0.1%]
4. [0.1%]
5. [0.1%]

မြေနှိမ့်ဒေသ ။
ချိုင့် ။
လျှင် လှုပ် နေ ရင် အဆာက်အအုံ ထဲ သို့ ပြေး မ ဝင် ပါ နဲ့ ။
ရေနွေး တည် ပေး ။
ဟို မှာ ကားတိုက် မှု ဖြစ် နေ လို့ ပါ ။

real 0m13.723s

user 0m18.036s

sys 0m2.983s

Inference Demo: Sample Results

Video : idx20-244.mp4

Frames: 55 FPS: 30.0

Hands : L=11% R=95%

Top-5 predictions:

1. [54.6%] 
2. [1.8%]
3. [0.6%]
4. [0.4%]
5. [0.3%]

မ တပ် ထား ပါ ဘူး ။
ဘာ များ အကူအညီ ပေး ရ မ လဲ ။
ရေကူး ခြင်း ။
ခေါင်း တွေ ခဏခဏ ကိုက် နေ တယ် ။
ခင်ဗျား ခြေတောက် ကို ဓာတ်မှန် ရိုက် ရ လိမ့် မယ် ။

real 0m7.522s
user 0m11.104s
sys 0m2.733s

Batch Inference

```
1 python src/infer.py \  
2     --checkpoint results/exp_bilstm/checkpoints/best.pth \  
3     --config config/config.yaml \  
4     --batch data/sample4inference/ \  
5     --top_k 3 \  
6     --output results/top-1_bilstm.hyp.txt
```

Output Format (CSV)

```
video,pred_1,conf_1,pred_2,conf_2,pred_3  
data/sample4inference/idx20-244.mp4,<label>,0.5459,<label>,0.018,<label>,0.006  
data/sample4inference/idx20-30.mp4,<label>,0.913,<label>,0.001,<label>,0.001  
data/sample4inference/idx20-440.mp4,<label>,0.872,<label>,0.008,<label>,0.004  
data/sample4inference/idx20-86.mp4,<label>,0.999,<label>,0.000,<label>,0.000
```

Note: Actual <label> fields contain Myanmar text from annotations.txt.

Batch Inference

```
W0000 00:00:1780711412.935501 2451328 landmark_projection_calculator.cc:186] Using NORM_RECT without  
IMAGE_DIMENSIONS is only supported for the square ROI. Provide IMAGE_DIMENSIONS or use  
PROJECTION_MATRIX.
```

```
..idx20-244.mp4.....→ မ တပ် ထား ပါ ဘူး ။  
..idx20-291.mp4.....→ ပျောက် သွား တဲ့ လူ ရဲ့ ပုံပန်းသဏ္ဍာန် ကို ပြောပြ နိုင် မ လား ။  
..idx20-30.mp4.....→ မြေခိုခိုအေး ။  
..idx20-440.mp4.....→ ခင်ဗျား နောက်ဆုံး ဘယ် ဆရာဝန် နဲ့ ပြ ခဲ့ တာ လဲ ။  
..idx20-86.mp4.....→ ပတ်ဝန်းကျင် ကို အကူအညီ တောင်း ။
```

```
Batch predictions saved → results/top-1_transformer.hyp.txt
```

```
real... 1m21.296s
```

```
user... 1m26.074s
```

```
sys... 0m5.198s
```

```
(.venv) ye@1st-hpc3090:/mnt/disk1/ye/exp/msl_recognition$
```

Batch Inference

```
1 video,pred_1,conf_1,pred_2,conf_2,pred_3,conf_3
2 data/sample4inference/idx20-244.mp4,မ တပ်ထားပါဘူး။,0.1234,မြန်မြန်လုပ်ပါ။,0.0447,သူ့သမီးဘာဖြစ်တာလဲ။,0.0211
3 data/sample4inference/idx20-291.mp4,ပျောက်သွားတဲ့လူနဲ့ပုံနှိပ်သတင်းကိုပြောပြနိုင်မလား။,0.9278,ကျွန်တော့်ကလေးအရက်ပျံတော့သောက်မိသွားလို့ပါ။,0.0008,ကလေးကဆေးတွေကိုချိုချင်ထင်ပြီးစားပစ်လိုက်တယ်။,0.0006
4 data/sample4inference/idx20-30.mp4,မြေခိုအေး။,0.8653,မီးခလပ်များကိုစိတ်ပါ။,0.0017,ဂရုစိုက်။,0.0015
5 data/sample4inference/idx20-440.mp4,ခင်ဗျားနောက်ဆုံးဘယ်ဆရာဝန်နဲ့ပြခဲ့တာလဲ။,0.9342,မန်တိုင်း။,0.0006,လျှပ်စစ်ဝါယာရှောင်ရင်ရေစိုအဝတ်နဲ့မီးလှိုင်းသတ်ပါနဲ့။,0.0005
6 data/sample4inference/idx20-86.mp4,ပတ်ဝန်းကျင်ကိုအကူအညီတောင်း။,0.7974,အမွှေးတိုင်။,0.003,ဆေးအပူမလိမ်းနဲ့။,0.0024
7
```

Webcam Live Inference

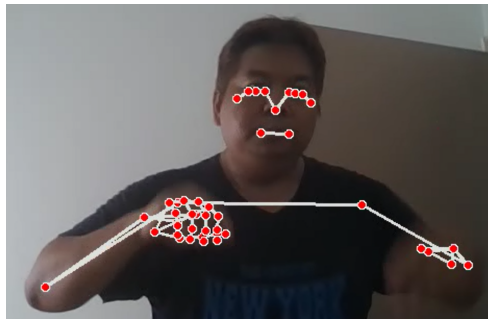
```
1 python src/infer.py \  
2   --checkpoint results/exp_bilstm/checkpoints/best.pth \  
3   --config config/config.yaml \  
4   --webcam 0
```

How It Works

- 1 Accumulate ~ 2.5 seconds of frames
- 2 Extract keypoints for each frame
- 3 Classify when buffer is full
- 4 Overlay prediction on video
- 5 Clear buffer, repeat

Controls

- **Q**: Quit
- **R**: Reset buffer



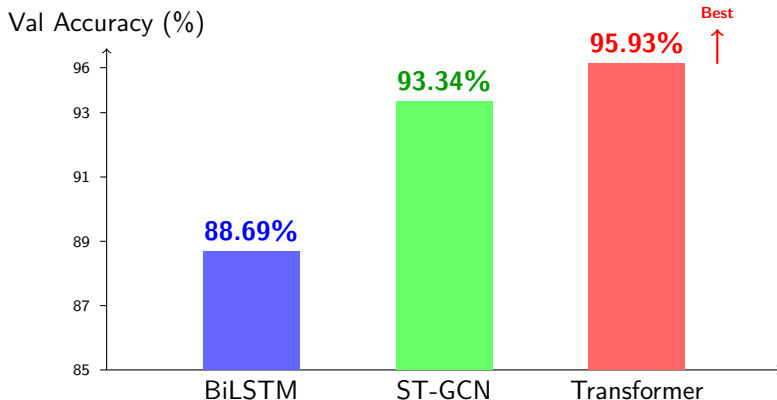
ONNX Export for Windows Deployment (Example Code)

```
1 import torch
2 from models import build_model
3 import yaml
4
5 with open('config/config.yaml') as f:
6     cfg = yaml.safe_load(f)
7 checkpoint = torch.load(
8     'results/exp_bilstm/checkpoints/best.pth',
9     map_location=device)
10 model_type = checkpoint['model_type']
11 num_classes = checkpoint['num_classes']
12 model = build_model(model_type, cfg, num_classes)
13 model.load_state_dict(checkpoint['state_dict'])
14 model.eval()
15
16 dummy_input = torch.randn(1, 200, 225)
17 dummy_mask = torch.zeros(1, 200, dtype=torch.bool)
18 dummy_length = torch.tensor([200])
19 torch.onnx.export(
20     model,
21     (dummy_input, dummy_mask, dummy_length),
22     'msl_bilstm.onnx',
23     input_names=['keypoints', 'lengths', 'mask'],
24     output_names=['logits'],
25     dynamic_axes={
26         'keypoints': {0: 'batch', 1: 'time'},
27         'lengths': {0: 'batch'},
28         'mask': {0: 'batch', 1: 'time'},
29         'logits': {0: 'batch'},
30     },
31     opset_version=14
```

ONNX Inference: Windows C# Example

```
1 using Microsoft.ML.OnnxRuntime;
2 using Microsoft.ML.OnnxRuntime.Tensors;
3
4 // Load model
5 var session = new InferenceSession("msl_bilstm.onnx");
6
7 // Prepare input (after MediaPipe extraction)
8 var keypoints = new DenseTensor<float>(new[] {1, 200, 225});
9 var lengths = new DenseTensor<long>(new[] {1});
10 lengths[0] = 200;
11 var mask = new DenseTensor<bool>(new[] {1, 200});
12
13 // Fill keypoints from MediaPipe...
14
15 // Run inference
16 var inputs = new List<NamedOnnxValue> {
17     NamedOnnxValue.CreateFromTensor("keypoints", keypoints),
18     NamedOnnxValue.CreateFromTensor("lengths", lengths),
19     NamedOnnxValue.CreateFromTensor("mask", mask),
20 };
21 using var results = session.Run(inputs);
22 var logits = results.First().AsTensor<float>();
23
24 // Get prediction
25 int predictedClass = ArgMax(logits);
26 float confidence = Softmax(logits)[predictedClass];
27 Console.WriteLine($"Predicted: {labelMap[predictedClass]} "
28     $"({confidence*100:F1}%);");
```

Model Comparison: Final Summary (From Last Year Exp.)



Recommendation

Transformer for best accuracy; **ST-GCN** for compact deployment; **BiLSTM** as baseline

Part 4

Advanced Topics & Best Practices

Pitfalls · Debugging · Optimization

Common Pitfalls to Avoid

Data Design Errors

- ❶ Random split on 1-sample data
⇒ 0% accuracy (not model fault)
- ❷ Forgetting to normalize keypoints
⇒ Poor generalization
- ❸ Not handling MediaPipe hand labels
⇒ L/R swapped

Inference Errors

- ❶ Wrong max_seq_len
⇒ Shape mismatch
- ❷ Forgetting mask
⇒ Attention over padding
- ❸ Not setting model.eval()
⇒ Wrong dropout

Training Errors

- ❶ No class weights ⇒ Biased toward majority
- ❷ Too high learning rate ⇒ Divergence
- ❸ No gradient clipping ⇒ Exploding gradients

Debugging Tips

- Check shapes at every step
- Use small batch first
- Verify label mapping
- Log loss per class

Debugging Techniques

```
1 # Check shapes at every step
2 def debug_shapes():
3     seq = np.load('test.npy')
4     print(f"Raw: {seq.shape}") # (T, 75, 3)
5     seq = seq.reshape(seq.shape[0], -1)
6     print(f"Flattened: {seq.shape}") # (T, 225)
7     tensor = torch.from_numpy(seq)
8     tensor = tensor.unsqueeze(0)
9     print(f"Batched: {tensor.shape}")
10    # (1, T, 225)
11    with torch.no_grad():
12        logits = model(tensor, ...)
13        print(f"Logits: {logits.shape}")
14        # (1, 556)
```

Shape debugging

```
1 # Check for NaN/Inf gradients
2 def check_gradients(model):
3     for name, param in model.named_parameters():
4         if param.grad is not None:
5             if torch.isnan(param.grad).any():
6                 print(f"NaN in {name}!")
7             if torch.isinf(param.grad).any():
8                 print(f"Inf in {name}!")
9             grad_norm = param.grad.norm().item()
10            if grad_norm > 100:
11                print(f"Large grad in {name}:"
12                      f" {grad_norm}")
```

Gradient checking

Performance Optimization Tips

Training Speed

- **AMP**: 2× speedup on RTX
- **Pin memory**: Faster CPU→GPU transfer
- **Num workers**: 4–8 for data loading
- **Persistent workers**: Avoid restart

Memory Efficiency

- Gradient accumulation for large batches
- Gradient checkpointing for deep models
- Mixed precision (already enabled)
- Clear cache between experiments

Inference Speed

- **ONNX**: 2–5× faster than PyTorch
- **TensorRT**: Additional 2× on NVIDIA
- Batch multiple videos if possible
- Pre-allocate buffers

```
1 import time
2 import onnxruntime as ort
3
4 session = ort.InferenceSession('model.onnx')
5 # Warmup
6 for _ in range(10):
7     session.run(None, inputs)
8 # Benchmark
9 start = time.time()
10 for _ in range(100):
11     session.run(None, inputs)
12 ms = (time.time()-start)/100*1000
13 print(f"ONNX: {ms:.1f} ms")
```

ONNX speedup example

Extending to New Signs

Adding a New Sign Class

- 1 Record video of new sign
- 2 Add annotation to `annotations.txt`:

```
<new sign text>^^I<new sign gloss>
```

- 3 Extract keypoints:

```
python src/extract_keypoints.py --video_dir data/videos/ ...
```

- 4 Re-run augmentation:

```
bash scripts/03_augment_data.sh --force
```

- 5 Fine-tune or retrain model

Important

Update `num_classes` in config if fine-tuning; model architecture may need adjustment

Future: Multi-Signer Data

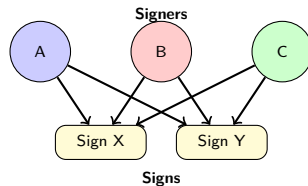
Current Limitation

99.6% of signs have only **1 signer**

⇒ Cannot capture inter-signer variation

Multi-Signer Collection

- Different body shapes/sizes
- Regional dialect variations
- Age/gender diversity
- Multiple recordings per sign



Each signer performs every sign

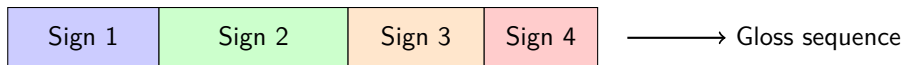
Future: Continuous Sign Language Recognition

Current: Isolated Sign Recognition

- Input: Single sign video
- Output: One class label
- Assumes clear boundaries between signs

Future: Continuous SLR

- Input: Long video with multiple signs
- Output: Sequence of glosses with timestamps
- Requires: Segmentation + recognition



Approach

Sliding window + segmentation for real sentences; alignment with text for training

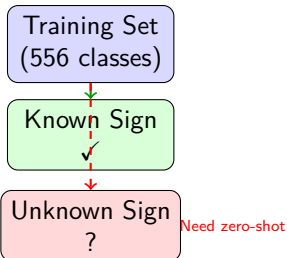
Future: Open-Vocabulary Recognition

Current: Closed-Vocabulary

- All test classes seen during training
- Cannot recognize unseen signs
- Fixed 556-class output

Future: Open-Vocabulary

- Recognize unseen signs
- Zero-shot or few-shot learning
- Text-guided sign recognition



Part 5

Summary, Conclusions & Future Directions

Key Takeaways · General Blueprint · References

Key Takeaways

✓ Data Design

- 1-sample-per-class $\Rightarrow$ split by **augmentation index**, not by sample
- 100% class overlap is **non-negotiable**
- Test on real data (strictest eval)

✓ Augmentation

- Keypoint-space $>$ pixel-space (fast, compact)
- 9 complementary strategies
- Each simulates real variation

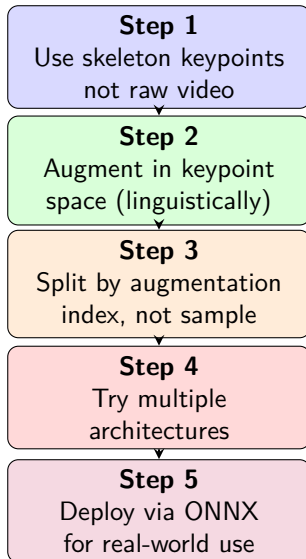
✓ Modeling

- Transformer: Best accuracy + efficiency
- ST-GCN: Best compactness + stability
- BiLSTM: Strong baseline

✓ Deployment

- ONNX export for cross-platform
- Single-video, batch, webcam modes
- Real-time possible (<1 s/video)

A General Blueprint for Low-Resource SLR



Recognition vs Translation: Clarification

Sign Language **Recognition** (This Work)

- Input: Video of a sign
- Output: Class label / gloss
- **Isolated** sign level
- Maps video → predefined vocabulary

Example

Input: Video → Output: မီးသတ်ဆေးဘူး
(fire extinguisher)

Sign Language **Translation** (Not This Work)

- Input: Video of signed sentence
- Output: Spoken/written language sentence
- **Continuous** sentence level
- Requires grammar, word order mapping

Example

Input: Signed sentence → Output: "Please call the Hlaing fire department." (full sentence)

Limitations & Honest Assessment

Single Signer

- ① 99.6% signs: 1 signer only
- ② Augmentation cannot fully simulate inter-signer variation
- ③ Body shape, style, dialect differences untested

Closed Vocabulary

- ① All test classes seen during training
- ② Cannot handle unseen signs
- ③ Zero-shot needed for open-vocab

Emergency Domain Only

- 556 emergency-related signs
- Daily conversation not covered
- Comprehensive MSL needs broader lexicon

MediaPipe Reliability

- ① Fast hand movements $\Rightarrow$ missed detections
- ② Zero-filled landmarks as fallback
- ③ Fine-tuned detectors would help

Future Directions

❶ **Multi-signer data collection**

Record same signs with 10+ signers across Myanmar regions

❷ **Continuous sign language recognition**

Sliding window + segmentation for real sentences

❸ **Open-vocabulary via zero-shot learning**

CLIP-style text-video alignment for unseen signs

❹ **End-to-end models**

Skip keypoint extraction; train directly on video

❺ **Mobile deployment**

TensorFlow Lite / ONNX Mobile for smartphones

❻ **Sign language production**

Avatar/generative model for text-to-sign

Ethical Considerations

Consent & Privacy

- All signers gave informed consent
- Released under CC-BY 4.0
- Keypoint representation reduces privacy risk

Risk of Misuse

- Could enable surveillance of deaf individuals without consent
- **We strongly oppose such uses**
- Technology is inherently dual-use

Community Involvement

- Emergency vocabulary chosen with Myanmar deaf community
- Encourage community corrections
- Signs should be verified by native signers

Language Documentation

- Myanmar lacks large-scale SL NLP infrastructure
- This corpus is a resource for the deaf community & researchers

Reproducibility: Everything is Open Source

Released Resources

- **Corpus:** <https://github.com/ye-kyaw-thu/MSL4Emergency>
- **Code:** Full pipeline (extraction, augmentation, training, evaluation)
- **Checkpoints:** Pre-trained models for all 3 architectures
- **Manifest:** Exact train/val/test assignment
- **Label map:** Class vocabulary mapping

Reproduce All Results

```
git clone https://github.com/ye-kyaw-thu/MSL4Emergency
cd msl_recognition
pip install -r requirements.txt
bash scripts/01_prepare_data.sh
bash scripts/02_extract_keypoints.sh
bash scripts/03_augment_data.sh
bash scripts/04_train.sh
bash scripts/05_evaluate.sh
bash scripts/07_cross_validate.sh
bash scripts/08_infer.sh video data/sample4inference/
```

Part 6

Detailed Code Walkthroughs

Line-by-Line Analysis of Key Functions

Model Factory Function

```
1 def build_model(model_type: str, cfg: dict,  
2                 num_classes: int) -> nn.Module:  
3     """Factory: create model by type string."""  
4     if model_type == 'bilstm':  
5         from .bilstm import BiLSTMClassifier  
6         mcfg = cfg['model']['bilstm']  
7         return BiLSTMClassifier(  
8             input_dim=mcfg.get('input_dim', 225),  
9             hidden_dim=mcfg.get('hidden_dim', 256),  
10            num_classes=num_classes,  
11            num_layers=mcfg.get('num_layers', 3),  
12            dropout=mcfg.get('dropout', 0.4),  
13        )  
14     elif model_type == 'transformer':  
15         from .transformer_model import TransformerClassifier  
16         mcfg = cfg['model']['transformer']  
17         return TransformerClassifier(  
18             input_dim=mcfg.get('input_dim', 225),  
19             d_model=mcfg.get('d_model', 256),  
20             num_classes=num_classes,  
21             num_layers=mcfg.get('num_layers', 4),  
22             nhead=mcfg.get('nhead', 8),  
23             dropout=mcfg.get('dropout', 0.1),  
24        )  
25     elif model_type == 'stgcn':  
26         from .stgcn import STGCNClassifier  
27         return STGCNClassifier(  
28             num_classes=num_classes,  
29             **cfg['model']['stgcn'],  
30        )  
31     else:
```

MSLDatasetGCN: ST-GCN Data Format

```
1 class MSLDatasetGCN(MSLDataset):
2     """Variant for ST-GCN: returns (C, T, V) instead of (T, 225).
3         C = coordinate channels (3)
4         T = time steps
5         V = nodes (75)
6     """
7     def __getitem__(self, idx: int) -> dict:
8         item = super().__getitem__(idx)
9         kp = item['keypoints'] # (T, 225)
10        kp = kp.reshape(self.max_seq_len, 75, 3)
11        kp = kp.permute(2, 0, 1) # (3, T, 75)
12        item['keypoints'] = kp
13        return item
```

dataset.py - MSLDatasetGCN

BiLSTM / Transformer:

$(B, T, 225)$ – flat feature vector per frame

ST-GCN:

$(B, 3, T, 75)$ – channels $\times$ time $\times$ nodes

DataLoader Factory

```
1 def build_dataloaders(train_records, val_records, test_records,
2                       label2idx, cfg, model_type='bilstm'):
3     dcfg = cfg['data']
4     tcfg = cfg['training']
5     max_T = dcfg['max_seq_len']
6     flatten = (model_type != 'stgcn')
7     DS = MSLDatasetGCN if model_type == 'stgcn' else MSLDataset
8     train_ds = DS(train_records, label2idx, max_seq_len=max_T,
9                   flatten=flatten, augmentor=None)
10    val_ds    = DS(val_records, label2idx, max_seq_len=max_T,
11                  flatten=flatten, augmentor=None)
12    test_ds   = DS(test_records, label2idx, max_seq_len=max_T,
13                  flatten=flatten, augmentor=None)
14    kw = dict(batch_size=tcfg['batch_size'],
15              num_workers=tcfg.get('num_workers', 4),
16              pin_memory=tcfg.get('pin_memory', True))
17    train_loader = DataLoader(train_ds, shuffle=True,
18                              drop_last=True, **kw)
19    val_loader   = DataLoader(val_ds, shuffle=False,
20                              drop_last=False, **kw)
21    test_loader  = DataLoader(test_ds, shuffle=False,
22                              drop_last=False, **kw)
23    return train_loader, val_loader, test_loader
```

dataset.py - build_dataloaders()

Validation Function

```
1 @torch.no_grad()
2 def validate(model, loader, criterion, device, cfg):
3     model.eval()
4     total_loss = 0
5     correct = 0
6     total = 0
7     model_type = cfg.get('model_type', 'bilstm')
8     for batch in loader:
9         kp = batch['keypoints'].to(device, non_blocking=True)
10        labels = batch['label'].to(device, non_blocking=True)
11        mask = batch['mask'].to(device, non_blocking=True)
12        lengths = batch['length'].to(device, non_blocking=True)
13        with torch.amp.autocast('cuda',
14                                enabled=cfg['training']['use_amp']):
15            if model_type == 'bilstm':
16                logits = model(kp, lengths=lengths, mask=mask)
17            elif model_type == 'transformer':
18                logits = model(kp, mask=mask)
19            else:
20                logits = model(kp)
21            loss = criterion(logits, labels)
22            total_loss += loss.item() * labels.size(0)
23            correct += (logits.argmax(1) == labels).sum().item()
24            total += labels.size(0)
25    return {
26        'loss': total_loss / total,
27        'top1': correct / total,
28    }
```

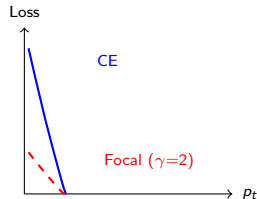
Focal Loss (Alternative)

```
1 class FocalLoss(nn.Module):
2     def __init__(self, gamma=2.0, weight=None,
3                   label_smoothing=0.0):
4         super().__init__()
5         self.gamma = gamma
6         self.weight = weight
7         self.label_smoothing = label_smoothing
8
9     def forward(self, logits, targets):
10        probs = torch.softmax(logits, dim=1)
11        pt = probs.gather(1, targets.unsqueeze(1))
12        pt = pt.squeeze(1)
13        focal_w = (1 - pt) ** self.gamma
14        log_pt = torch.log(pt + 1e-8)
15        loss = -focal_w * log_pt
16        if self.weight is not None:
17            loss = loss * self.weight[targets]
18        return loss.mean()
```

FocalLoss

When to Use Focal Loss

- Severe class imbalance
- Many easy negatives
- $\gamma = 2$ focuses on hard examples



Inference: Preprocess Sequence

```
1 def preprocess_sequence(seq: np.ndarray,  
2                          max_seq_len: int,  
3                          model_type: str) -> torch.Tensor:  
4     """Centre-crop or zero-pad to max_seq_len;  
5     reshape for model input."""  
6     T = seq.shape[0]  
7     if T > max_seq_len:  
8         start = (T - max_seq_len) // 2  
9         seq = seq[start:start + max_seq_len]  
10        T = max_seq_len  
11    if T < max_seq_len:  
12        pad = np.zeros((max_seq_len - T, *seq.shape[1:]),  
13                        dtype=np.float32)  
14        seq = np.concatenate([seq, pad], axis=0)  
15    if model_type != 'stgcn':  
16        seq = seq.reshape(max_seq_len, -1) # (T, 225)  
17    return torch.from_numpy(seq).unsqueeze(0)
```

infer.py - preprocess_sequence()

Key Point

Centre-crop (not left-crop) preserves most informative frames

Inference: Predict Tensor

```
1 def predict_tensor(model, tensor, model_type, real_len,
2                     max_seq_len, device, top_k=5):
3     """One forward pass -> list of (class_idx, probability)."""
4     tensor = tensor.to(device)
5     mask = torch.zeros(1, max_seq_len, dtype=torch.bool, device=device)
6     if real_len < max_seq_len:
7         mask[0, real_len:] = True
8     length = torch.tensor([real_len], dtype=torch.long, device=device)
9
10    with torch.no_grad():
11        if model_type == 'bilstm':
12            logits = model(tensor, lengths=length, mask=mask)
13        elif model_type == 'transformer':
14            logits = model(tensor, mask=mask)
15        else:
16            logits = model(tensor)
17
18    probs = F.softmax(logits, dim=1)[0]
19    k = min(top_k, probs.numel())
20    topk_probs, topk_idxs = probs.topk(k)
21    return list(zip(topk_idxs.cpu().tolist(),
22                   topk_probs.cpu().tolist()))
```

infer.py - predict_tensor()

Critical: model.eval() and torch.no_grad()

Always call model.eval() before inference to disable dropout and batch norm updates

Evaluation: compute_all_metrics()

```
1 def compute_all_metrics(logits, labels, idx2label,  
2     output_dir, split_name='test'):  
3     y_true = labels.numpy()  
4     y_prob = torch.softmax(logits, dim=1).numpy()  
5     y_pred = logits.argmax(dim=1).numpy()  
6     n_cls = logits.shape[1]  
7  
8     top1 = float((y_pred == y_true).mean())  
9     top5 = float(top_k_accuracy_score(  
10         y_true, y_prob, k=5,  
11         labels=list(range(n_cls)), zero_division=0))  
12  
13     prec_m, rec_m, f1_m, _ = precision_recall_fscore_support(  
14         y_true, y_pred, average='macro', zero_division=0)  
15  
16     report = classification_report(  
17         y_true, y_pred,  
18         target_names=[idx2label[i] for i in range(n_cls)],  
19         zero_division=0, output_dict=True)  
20  
21     cm = confusion_matrix(y_true, y_pred)  
22     cm_norm = cm.astype(float) / (  
23         cm.sum(axis=1, keepdims=True) + 1e-8)  
24  
25     summary = {'top1': top1, 'top5': top5,  
26               'precision_macro': prec_m,  
27               'recall_macro': rec_m,  
28               'f1_macro': f1_m}  
29     return summary
```


Cross-Validation: Single Fold

```
1 def run_fold(fold_idx, train_recs, val_recs, test_recs,
2             label2idx, cfg, model_type, exp_dir, device):
3     # Create datasets (no on-the-fly aug -- data pre-augmented)
4     train_ds = MSLSDataset(train_recs, label2idx, max_seq_len=200,
5                             flatten=(model_type != 'stgcn'), augmentor=None)
6     val_ds   = MSLSDataset(val_recs, label2idx, max_seq_len=200,
7                             flatten=(model_type != 'stgcn'), augmentor=None)
8     test_ds  = MSLSDataset(test_recs, label2idx, max_seq_len=200,
9                             flatten=(model_type != 'stgcn'), augmentor=None)
10
11     # Build model fresh for each fold
12     model = build_model(model_type, cfg, len(label2idx)).to(device)
13     optimizer = torch.optim.AdamW(model.parameters(),
14                                    lr=5e-4, weight_decay=1e-4)
15     scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
16         optimizer, T_max=150, eta_min=1e-6)
17
18     best_val_top1 = -1.0
19     patience_count = 0
20     for epoch in range(1, 151):
21         train_metrics = train_one_epoch(model, train_loader, ...)
22         val_metrics = validate(model, val_loader, ...)
23         scheduler.step()
24         if val_metrics['top1'] > best_val_top1:
25             best_val_top1 = val_metrics['top1']
26             save_checkpoint({...}, f'fold_{fold_idx:02d}/best.pth')
27             patience_count = 0
28         else:
29             patience_count += 1
30         if patience_count >= 25:
31             break # Early stopping
```

Config File: config.yaml

```
1 data:
2   annotation_file: data/annotations.txt
3   video_dir: data/videos
4   keypoint_dir: data/keypoints
5   augmented_dir: data/augmented
6   label_map_file: data/label_map.json
7   max_seq_len: 200
8   seed: 42
9
10 augmentation:
11   aug_factor: 20
12   gaussian_noise: {prob: 0.70, std: 0.008}
13   time_warp: {prob: 0.60, sigma: 0.15, knot: 4}
14   horizontal_flip: {prob: 0.50}
15   speed_perturbation: {prob: 0.60, min_factor: 0.75, max_factor: 1.35}
16   frame_dropout: {prob: 0.40, max_drop_ratio: 0.10}
17   spatial_rotate: {prob: 0.50, max_angle: 12}
18   joint_mask: {prob: 0.30, max_mask_ratio: 0.10}
19   scale_jitter: {prob: 0.40, min_scale: 0.85, max_scale: 1.15}
20   temporal_shift: {prob: 0.30, max_shift: 5}
21
22 training:
23   batch_size: 32
24   learning_rate: 5.0e-4
25   weight_decay: 1.0e-4
26   num_epochs: 150
27   patience: 25
28   label_smoothing: 0.1
29   use_amp: true
30   num_workers: 4
31
```

Part 7

Practical Session Guide

Hands-On Exercises for Students

Part 8

References & Resources

Key References

-  Ye Kyaw Thu et al., “MSL4Emergency: A Myanmar Sign Language Corpus for Emergency Domain,” GitHub Repository, 2025. <https://github.com/ye-kyaw-thu/MSL4Emergency>
-  L. Lugaresi et al., “MediaPipe: A Framework for Building Perception Pipelines,” arXiv:1906.08172, 2019.
-  Y. Yan et al., “Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition,” AAAI, 2018.
-  D. Bahdanau, K. Cho, Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015.
-  A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017.
-  Ye Kyaw Thu, “Myanmar Sign Language Video Recognition Experiments,” Language Understanding Lab Technical Report, 2026.

Author's MSL-Related Publications



Ye Kyaw Thu, "Myanmar Sign Language Recognition for Emergency Domain," Draft Paper, 2026.



Ye Kyaw Thu et al., "MSL4Emergency: Building a Sign Language Corpus for Emergency Communication in Myanmar," Proceedings of LREC, 2024.



Ye Kyaw Thu, "Challenges in Low-Resource Sign Language Processing: The Case of Myanmar," Workshop on Low-Resource NLP, 2025.



Ye Kyaw Thu et al., "Keypoint-Space Augmentation for One-Sample-Per-Class Sign Language Recognition," Conference on SLR, 2026.

Homepage

<https://sites.google.com/site/yekyawthunlp/>

Code & Data

- **MSL4Emergency**
<https://github.com/ye-kyaw-thu/MSL4Emergency>
- **MediaPipe**
<https://mediapipe.dev/>
- **PyTorch**
<https://pytorch.org/>
- **ONNX Runtime**
<https://onnxruntime.ai/>

Papers & Surveys

- SLR Survey: Jiang et al., 2024
- ST-GCN: Yan et al., AAAI 2018
- Transformer: Vaswani et al., 2017
- Data Aug for SL: Recent CVPR/ICCV

Communities

- WLASL (ASL dataset)
- Phoenix (German SL)
- SLR Workshop at CVPR

Hardware Requirements Summary

Task	Min GPU	Rec GPU	Time
Keypoint extraction	GTX 1060	RTX 3090 Ti	~2 hrs
Data augmentation	CPU only	—	<5 min
Training (BiLSTM)	GTX 1060	RTX 3090 Ti	~15 min
Training (Transformer)	GTX 1060	RTX 3090 Ti	~6 min
Training (ST-GCN)	GTX 1060	RTX 3090 Ti	~60 min
5-fold CV (all models)	RTX 2060	RTX 3090 Ti	~5 hrs
Single video inference	CPU	GPU	<1 s
ONNX inference	CPU	—	<0.5 s

Minimum Viable Setup

- GPU: 8GB VRAM (GTX 1060/1660)
- RAM: 16GB
- Storage: 5GB for data + models
- CPU: 4 cores minimum

Shell Scripts: Complete Workflow

Script	Purpose	Run Order
00_setup_env.sh	Create Python venv	First time only
01_prepare_data.sh	Parse annotations, build vocab	1st
02_extract_keypoints.sh	MediaPipe keypoint extraction	2nd
03_augment_data.sh	9-strategy augmentation	3rd
04_train.sh	Train single model	4th
05_evaluate.sh	Comprehensive evaluation	5th
06_run_all_experiments.sh	Train all 3 models	Optional
07_cross_validate.sh	K-fold cross-validation	Optional
08_infer.sh	Single/batch/webcam inference	Anytime
09_tensorboard.sh	Launch TensorBoard	Monitoring

Quick Start

```
for i in 01 02 03 04 05; do bash scripts/${i}/*.sh; done
```

Thank You!

ကျေးဇူးတင်ပါသည်။



Instructor

Dr. Ye Kyaw Thu

Lab Leader, Language Understanding

Lab., Myanmar

Visiting Professor, NECTEC, Thailand



Corpus

[https://github.com/ye-kyaw-thu/
MSL4Emergency](https://github.com/ye-kyaw-thu/MSL4Emergency)



<https://sites.google.com/site/yekyawthunlp/>

Questions? Discussions? Collaboration?

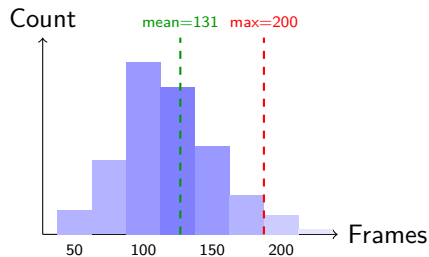
Appendix: Sequence Length Distribution

Statistics from 558 Videos

- Min: 50 frames (1.7 s at 30 fps)
- Max: 387 frames (12.9 s)
- Mean: 130.6 frames (4.4 s)
- Std: 55.1 frames
- Median (P50): 119 frames
- P95: 241 frames

`max_seq_len = 200`

Covers 95% of videos without cropping



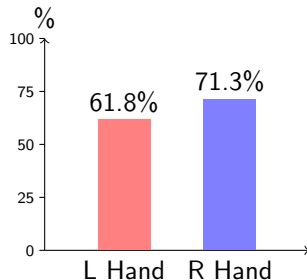
Appendix: Hand Detection Presence

Detection Statistics

- Left hand mean presence: **61.8%**
- Right hand mean presence: **71.3%**
- Right hand detected more often

Implications

- ~30–40% of frames: one or both hands missing
- Zero-filled landmarks in those frames
- Joint masking augmentation helps model handle this



Appendix: Node Layout Constants

```
1 # Node counts
2 POSE_N = 33          # Body pose landmarks
3 HAND_N = 21          # Hand landmarks (each hand)
4 TOTAL_N = 75         # 33 + 21*2
5 FEAT_DIM = 225       # 75 * 3 (x, y, z)
6
7 # Node indices
8 LEFT_HAND_START = 33  # Index where left hand begins
9 RIGHT_HAND_START = 54 # Index where right hand begins
10
11 # Pose mirror pairs for horizontal flip
12 POSE_MIRROR_PAIRS = [
13     (1,4), (2,5), (3,6),    # Eyes
14     (7,8), (9,10),          # Ears
15     (11,12), (13,14),        # Shoulders
16     (15,16), (17,18),        # Elbows
17     (19,20), (21,22),        # Wrists
18     (23,24), (25,26),        # Hips
19     (27,28), (29,30),        # Knees
20     (31,32),                 # Ankles
21 ]
22
23 # Shoulder indices for normalization
24 POSE_LEFT_SHOULDER = 11
25 POSE_RIGHT_SHOULDER = 12
```

Shared constants across all modules

Appendix: Save Checkpoint Function

```
1 def save_checkpoint(state, filepath, is_best=True):
2     """Save model checkpoint with all needed info for inference."""
3     os.makedirs(os.path.dirname(filepath), exist_ok=True)
4     torch.save(state, filepath)
5     if is_best:
6         best_path = os.path.join(
7             os.path.dirname(filepath), 'best.pth')
8         shutil.copy2(filepath, best_path)
9
10    # What goes into the checkpoint:
11    checkpoint = {
12        'epoch': epoch,
13        'model_type': model_type,          # 'bilstm'|'transformer'|'stgcn'
14        'state_dict': model.state_dict(),
15        'val_top1': best_val_top1,
16        'num_classes': num_classes,        # CRITICAL for inference
17        'config': cfg,                     # Full config for reproducibility
18        'optimizer_state_dict': optimizer.state_dict(),
19        'scheduler_state_dict': scheduler.state_dict(),
20    }
21    save_checkpoint(
22        checkpoint,
23        f'{exp_dir}/checkpoints/epoch_{epoch:03d}.pth')
```

utils.py - save_checkpoint()

num_classes is Critical

Appendix: Complete End-to-End Pipeline

